

인공지능기초 - 컴퓨터공학과

기말 프로젝트

기업 투자 및 경영 관계 네트워크

Hybrid Retrieval + Graph RAG

기업 투자 및 경영 관계 네트워크 기반 Hybrid Retrieval + Graph RAG 시스템

과목명	기계학습
학적	경영학과
	202320387
이름	정민성

깃허브 주소

https://github.com/wjdalstjd411-alt/202320387_-_15_-_/tree/main/new%20project

<인공지능기초 15주차 기말대체과제>

기업 투자 및 경영 관계 네트워크 기반 Hybrid
Retrieval + Graph RAG 시스템

학과: 경영학과
학번: 202320387
이름: 정민성

목차

1. 서론

2. 배경지식

3. 시스템 설계

4. 구현상세

5. 결과 및 분석

6. 고찰

1.1 프로젝트 배경

최근 대규모 언어 모델(LLM)의 발전과 함께 RAG는 외부 지식을 활용하여 보다 정확한 답변을 생성하는 대표적인 방법으로 활용되고 있다. 기존 LLM은 학습 시점 이후의 정보를 반영하지 못하며, 사실과 다른 내용을 생성하는 AI환각 문제가 잔존한다. 이러한 문제상황에서 RAG는 외부 문서를 검색한 뒤 해당 정보를 기반으로 답변을 생성함으로써 이러한 한계를 완화할 수 있다.

그러나 RAG 역시 여러 한계를 가지고 있다. 첫째, 검색 단계에서 적절한 문서를 찾지 못할 경우 **답변 품질이 크게 저하**된다. 둘째, 관련 문서를 검색하더라도 LLM이 이를 잘못 해석하여 **잘못된 정보의 환각을 생성**할 수 있다. 셋째, 벡터 기반 검색은 문서 간 의미적 유사성을 찾는 데 강점을 가지지만 엔티티 간 관계를 명시적으로 표현하지 못하기 때문에 **다단계 추론에 한계**가 존재한다.

11주차와 13주차 실습에서 구현한 기본 RAG 시스템은 문서를 벡터화하고 검색 결과를 기반으로 답변을 생성하는 구조였다. 하지만 해당 시스템은 **검색 품질을 객관적으로 측정하기 어렵고, 복잡한 관계 질의에 대한 답변 능력이 제한적이라는 문제**가 있었다.

이에 이번 프로젝트에서는 두 가지 방향으로 기존 RAG를 확장하였다.

첫 번째는 **평가 가능성**이다. **RAGas**를 활용하여 검색 및 답변 품질을 정량적으로 평가하고, 추가적으로 retrieval 간 응답시간 차이를 분석하는 **커스텀 평가 지표**를 도입하였다.

두 번째는 **구조화된 지식의 활용**이다. Neo4j 기반 Knowledge Graph를 구축하고 GraphCypherQChain을 이용하여 **자연어 질문을 그래프 질의로 변환**함으로써 관계 기반 질의와 다단계 추론을 수행할 수 있도록 하였다.

본 프로젝트의 목표는 Dense Retrieval, Sparse Retrieval, Hybrid Retrieval, Graph RAG를 구현하고 각각의 특징과 성능을 비교·분석하는 것이다.

*이번 프로젝트는 대부분 LLM을 통해 코드를 만들어 제작하고 설계, 오류 수정이나 패키지 설치 등을 제가 하였습니다. -> 그러하여 패키지 버전 오류로 데모 영상이 제대로 나오지 않은 점 죄송합니다..

1.2 프로젝트 목표

이번 프로젝트의 목표는 다음과 같다.

1. 다양한 Document Loader를 활용 -> 데이터를 적재하고 메타데이터 부여
2. Text Splitter 활용 -> 문서를 청크 단위로 분할
3. OpenAI Embedding 모델과 ChromaDB 이용 -> Vector RAG 구축.
4. Neo4j 활용 -> Knowledge Graph 구축, Graph RAG 구현
5. Dense Retrieval과 Hybrid Retrieval 성능 비교
6. RAGAS 평가와 커스텀 평가 지표, 아우라db 쿼리 그래프 활용 -> 시스템 성능 정량적 분석

2.배경 지식

2.1 Standard RAG Workflow

일반적인 RAG 시스템은 다음과 같은 5단계로 구성된다.

- 1.Document Loading: 먼저 원본 문서를 프로젝트 파일에 적재
- 2.Text Splitting: 적절한 크기의 청크로 분할
- 3.Embedding: 임베딩 모델을 이용해 각 청크를 벡터로 변환하고 벡터 데이터베이스에 저장
- 4.Retrieval: 사용자의 질문이 입력되면 질문 또한 벡터화하여 유사한 문서를 검색
- 5.Generation: 검색된 문서를 LLM에게 제공하여 최종 답변을 생성

2.2 Text Splitter와 의미 보존 전략

텍스트 split은 RAG 성능에 직접적인 영향을 미치는 핵심 과정이다.

-청크가 너무 작으면 -> 문맥 손실, / 너무 크면 -> 검색 정확도 감소.
따라서 적절한 크기의 청크를 생성하는 것이 중요.

이번 프로젝트에서는 RecursiveCharacterTextSplitter와 SemanticChunker를 사용하였다.

-Recursive Splitter: 문단, 문장, 단어 순으로 텍스트를 분할 -> 문맥 구조 최대한 유지

-Semantic Chunker: 임베딩 기반 의미 유사도 활용 -> 의미적으로 유사한 문장을 하나의 청크로 묶음.

이를 통해 단순 문자 길이 기준 분할보다 의미 보존 효과를 기대할 수 있다.

2.3 Dense Retrieval과 Sparse Retrieval

Sparse Retrieval:

-키워드 기반 검색 방식(ex.BM25)

-BM25: 사용자 질문에 포함된 단어가 문서에 얼마나 자주 등장하는지 계산 -> 관련도를 측정

= > 따라서 기업명이나 인물명과 같은 **고유명사 검색에 강점**을 가짐.

Dense Retrieval:

-임베딩 기반 검색 방식.

-텍스트를 고차원 벡터 공간에 매핑 -> 의미적으로 유사한 문서 검색

- **동일한 의미 가진 서로 다른 표현 검색할 수 있다는 장점**

-**But**) Dense Retrieval은 특정 키워드가 반드시 포함되어야 하는 질문에서는 성능 저하 가능성 있음.

==> 이를 보완하기 위해 Sparse Retrieval과 Dense Retrieval을 결합하여 한계를 보완한 **Hybrid Retrieval**이 활용된다

2.4 MMR(Maximal Marginal Relevance)

-기존 벡터 검색의 문제: 유사도가 높은 문서를 여러 개 반환하는 과정 -> **중복된 내용이 반복적으로 검색되는 문제 존재**

-MMR은 검색 정확도 + 다양성 사이의 균형 고려하는 방법

-검색 결과의 관련성 유지하면서도 이미 선택된 문서와 지나치게 유사한 문서는 제외

-> 다양한 정보 포함하는 검색 결과 제공

==> 이를 통해 동일한 내용이 반복적으로 검색되는 현상 줄일 수 있음

2.5 RAGAS 평가 지표

RAGAS:

- RAG 시스템 평가 위한 대표적인 프레임워크
- LLM-as-Judge 방식을 활용
- LLM이 평가자의 역할을 수행 -> 검색 결과 + 생성 답변 분석하고 점수 부여

*이번 프로젝트에서는 다음 네 가지 지표를 사용하였다.

- (1)**Faithfulness**: 생성된 답변이 검색된 문서에 얼마나 근거하고 있는가.
- (2)**Answer Relevancy**: 사용자의 질문과 얼마나 관련되어 있는가
- (3)**Context Precision**: 검색된 문서 중 실제로 유용한 문서 비율은 얼마인가
- (4)**Context Recall**: 정답 생성에 필요한 정보가 검색 결과에 얼마나 포함되었는가

2.6 Knowledge Graph와 Cypher

Knowledge Graph:

- 엔티티와 관계를 그래프 구조로 표현하는 지식 표현 방식
- 그래프는 노드 + 엣지로 구성
- Ex.
Han Jong-Hee → CEO_OF → Samsung Electronics
와 같은 형태로 관계를 명시적으로 표현 가능
- 관계 추적과 다단계 추론에 강점.

Cypher: Neo4j에서 사용하는 그래프 질의 언어

- SQL이 테이블 구조를 대상으로 하는 것과 달리 Cypher는 노드와 관계를 탐색한다.

Ex.

특정 기업의 CEO를 찾는 Cypher 질의는 다음과 같다.

```
MATCH (p)-[]->(c{name:"Samsung Electronics"})
```

```
RETURN p
```

==> 복잡한 관계 기반 질의를 효율적으로 수행할 수 있음.

3.시스템 설계

3.1 전체 시스템 아키텍처

본 프로젝트는 Vector RAG와 Graph RAG를 함께 구축하고 성능을 비교하는 것을 목표로 설계하였다.

전체 시스템은 다음과 같은 흐름으로 구성된다.

데이터 적재

- 문서 전처리 및 메타데이터 부여
- Text Splitting
- 임베딩 생성
- Vector Database 저장
- Dense / Sparse / Hybrid Retrieval
- LLM 답변 생성
- RAGAS 평가

한편 Graph RAG는 다음과 같은 흐름으로 구성된다.

CSV 데이터

- Neo4j Graph 구축
- GraphCypherQAChain
- 자연어 → Cypher 변환
- Graph Query 수행
- 답변 생성

두 시스템 모두 동일한 테스트셋을 활용하여 평가를 수행하였으며, Dense Retrieval과 Hybrid Retrieval의 검색 품질을 비교하였다.

3.2 Graph Schema 설계

Knowledge Graph는 기업, 인물, 대학교 간 관계를 표현하기 위해 설계하였다.
노드 타입은 다음과 같다.

Node Type	설명
Company	기업
Person	인물
University	대학교

관계 타입은 다음과 같다.

Edge Type	설명
CEO_OF	CEO 관계
GRADUATED_FROM	졸업 관계
WORKS_AT	근무 관계
FOUNDED	창업 관계

그래프 구조를 활용함으로써 단순 엔티티 검색뿐 아니라 관계 추적 및 다단계 추론이 가능하도록 설계하였다.

3.3 테스트셋 설계

RAGAS 평가를 위해 총 30개의 질문-정답 쌍(Test Set)을 직접 구축하였다.
질문은 특정 유형에 편중되지 않도록 다음과 같이 구성하였다.
(예시 질문 3개)

"question": "Samsung Electronics CEO는 누구인가?",
"ground_truth": "Han Jong-hee"
와 같은 유형 10개

"question": "Han Jong-hee는 어느 회사 CEO인가?",
"ground_truth": "Samsung Electronics"
와 같은 유형 10개

"question": "Samsung Electronics CEO는 어느 대학을 졸업했는가?",
"ground_truth": "Inha University"

와 같은 유형 10개

그리고 00기업의 자회사는 몇 개인가, A와B는 어떤 관계인가에 대한 질문에 답할 수 있도록 추가 설계하였다.

이와 같이 다양한 유형의 질문을 포함하여 Retrieval 및 Generation 성능을 종합적으로 평가하고자 하였다.

4.구현 상세

4.1 Document Loader와 메타데이터 설계

이번 프로젝트에서는 CSV 형식의 구조화된 데이터와 JSON 형식의 문서를 활용하였다. 데이터 적재를 위해 LangChain의 CSVLoader와 JSONLoader를 사용하였다.

```
from langchain_community.document_loaders import CSVLoader
from langchain_community.document_loaders import JSONLoader
import os
```

```
[LOADING] data/companies.csv
```

```
[SUCCESS] 20개 문서 로드
```

사용된 데이터: 기업 정보, 인물 정보, 대학교 정보, 관계 정보, 기업 설명 문서 등으로 구성.

각 문서는 코드 과정에서 source와 type 메타데이터를 추가 -> 데이터의 출처와 문서

```
[LOADING] data/people.csv
```

```
[SUCCESS] 20개 문서 로드
```

기업 데이터는 type="company", 인물 데이터는 type="person", 관계 데이터는

```
[LOADING] data/universities.csv
```

```
[SUCCESS] 15개 문서 로드
```

```
[SUCCESS] 20개 문서 로드
```

```
companies.csv
```

```
company_document.csv :S.csv
```

```
people.csv
```

```
relations.csv :SV
```

```
universities.csv
```

```
[LOADING] data/company_document.csv
```

```
[SUCCESS] 20개 문서 로드
```

```
[JSON] 30개 문서 로드
```

```
30
```

4.2 Metadata 부여

검색 품질 향상을 위해 모든 문서에 메타데이터를 추가하였다.

-source: 원본 파일 경로

-type: 문서 유형

이를 통해 검색 결과의 출처를 추적할 수 있으며, 특정 유형의 데이터만 필터링하는 검색도 가능하도록 설계하였다.

```
for doc in loaded_docs:
    doc.metadata["source"] = file

    if "companies" in file:
        doc.metadata["type"] = "company"

    elif "people" in file:
        doc.metadata["type"] = "person"

    elif "universities" in file:
        doc.metadata["type"] = "university"

    elif "relations" in file:
        doc.metadata["type"] = "relation"

    elif "company_document" in file:
        doc.metadata["type"] = "company_document"

docs.extend(loaded_docs)
```

4.3 Text Splitter

문서를 임베딩하기 전 적절한 크기의 청크로 분할하기 위해 RecursiveCharacterTextSplitter 와 SemanticChunker를 활용하였다.

RecursiveCharacterTextSplitter의 주요 설정:

-chunk_size = 500

-chunk_overlap = 50

chunk_size는 하나의 청크가 충분한 문맥 정보를 포함하면서도 임베딩 모델의 **효율적인 처리 범위를 유지할 수 있도록** 500으로 설정하였다. 또한 chunk_overlap은 인접한 청크 간 **문맥 단절을 완화**하기 위해 50으로 설정하였다

실험 결과:

Recursive Splitter:

- 185개 청크 생성
- 일정한 크기의 청크를 안정적으로 생성

Semantic Chunker:

- 166개 청크 생성
- 의미적으로 유사한 문장을 하나의 청크로 묶는 특징

=> 정량적으로는 Recursive Splitter가 더 많은 청크를 생성하였으나, Semantic Chunker는 문맥 보존 측면에서 장점을 보였다.

```
(venv) PS C:\Users\정민성\Desktop\new project> python test_splitter_compare.py

===== SPLITTER COMPARISON =====

원본 문서 수: 146
Recursive 청크 수: 185
Semantic 청크 수: 166

===== Recursive Example =====
company_id: C001
company_name: Samsung Electronics
industry: Electronics
founded_year: 1969

===== Semantic Example =====
company_id: C001
company_name: Samsung Electronics
industry: Electronics
founded_year: 1969
```

-임베딩 모델은 텍스트를 고차원 벡터로 변환하여, 의미적으로 유사한 문서가 가까운 위치에 배치되도록 함.

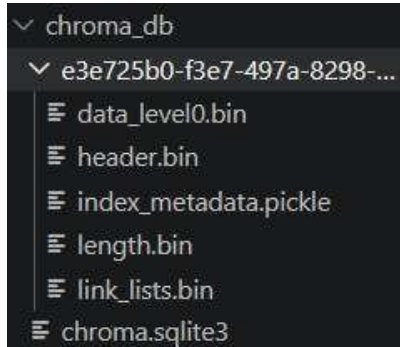
-생성된 임베딩 벡터는 ChromaDB에 저장 -> 벡터 검색에 활용

```
embeddings = OpenAIEmbeddings(
    model="text-embedding-3-small"
)
```

4.5 Vector Store 구축

벡터 데이터베이스로 ChromaDB를 사용하였다. 전처리된 문서를 임베딩한 후 ChromaDB에 저장하였으며, 사용자 질문이 입력되면 동일한 임베딩 공간에서 유사도 검색을 수행하도록 구현하였다.

ChromaDB: 빠른 검색 속도 + LangChain과의 높은 호환성을 제공 -> RAG 시스템 구축에 적합함.



4.6 Hybrid Retrieval

Dense Retrieval의 한계를 보완하기 위해 Hybrid Retrieval을 구현하였다.

Hybrid Retrieval은 벡터 기반 검색(Dense Retrieval)과 키워드 기반 검색(BM25 Retrieval)을 결합한 방식이다. 본 프로젝트에서는 Ensemble Retriever를 활용하여 두 검색 결과를 통합하였다.

이를 통해 의미 기반 검색의 장점과 키워드 검색의 장점을 동시에 활용할 수 있었으며, 특히 기업명이나 인물명과 같은 고유명사 검색에서 검색 성능 향상을 기대할 수 있었다.

이후 성능 평가 단계에서 Dense Retrieval과 Hybrid Retrieval의 RAGAS 점수를 비교하여 검색 품질 향상 여부를 분석하였다.

```
(venv) PS C:\Users\정민성\Desktop\new project> python test_hybrid.py

종 분서 수: 146
Failed to send telemetry event ClientStartEvent: capture() takes 1 positional argument but 3 were given
Failed to send telemetry event ClientCreateCollectionEvent: capture() takes 1 positional argument but 3 were given
Failed to send telemetry event CollectionQueryEvent: capture() takes 1 positional argument but 3 were given
=====
Result 1
company_name: Krafton
=====
Result 2
The company collaborates with international studios and technology partners to expand its intellectual property portfolio. Krafton continues to strengthen its position in the gaming industry through strategic investments, research initiatives, and global market expansion efforts.
Result 3
document: Krafton is a gaming company known for developing and publishing interactive entertainment products for global audiences. The company focuses on creating immersive gaming experiences through innovative design, advanced technology, and long term content development strategies. Krafton invests in artificial intelligence, game engines, virtual worlds, and next generation entertainment technologies. The company collaborates with international studios and technology partners to expand its
Result 4
document: SK Hynix is a semiconductor company specializing in memory chips and advanced electronic components. The company develops technologies that support computing, artificial intelligence, mobile devices, and cloud infrastructure. SK Hynix invests heavily in semiconductor research and manufacturing facilities. The company collaborates with global technology partners and contributes to innovation in data processing technologies. SK Hynix plays a critical role in the global semiconductor
Result 5
=====
Result 5
company invests in manufacturing expansion and next generation healthcare technologies. It plays an important role in supporting pharmaceutical supply chains and global medical research.
=====
Result 6
document: Samsung Biologics is a biotechnology company specializing in contract development and manufacturing services for pharmaceutical products. The company supports global healthcare organizations through advanced production facilities and research capabilities. Samsung Biologics focuses on quality management, regulatory compliance, and innovation in biotechnology. The company invests in manufacturing expansion and next generation healthcare technologies. It plays an
Result 7
company invests in artificial intelligence, automation technologies, and smart logistics systems. Samsung SDS is recognized for its expertise in large scale IT infrastructure and digital innovation projects.
Result 8
document: Naver is a major internet platform company in South Korea. The company operates search services, digital content platforms, and solutions, artificial intelligence technologies, and business services. Naver invests heavily in research and development to improve user experiences and support innovation. The company collaborates with startups, technology firms, and content creators. Naver continues to expand its influence through investments in digital transformation, e-commerce
```

4.7 Knowledge Graph RAG 구축: 그래프 데이터베이스 구축

기존 Vector RAG는 문서의 의미적 유사성을 기반으로 정보를 검색할 수 있지만, 엔티티 간 관계를 명시적으로 표현하거나 다단계, 연속 추론을 수행하는 데에는 한계가 존재한다. 이를 보완하기 위해 Neo4j를 활용한 Knowledge Graph를 구축하였다.

본 프로젝트에서는 기업(Company), 인물(Person), 대학교(University)를 주요 엔티티로 정의하였으며, CSV 데이터에 포함된 관계 정보를 이용하여 그래프를 생성하였다.

```
from neo4j import GraphDatabase

load_dotenv()

URI = os.getenv("NEO4J_URI")
USERNAME = os.getenv("NEO4J_USERNAME")
PASSWORD = os.getenv("NEO4J_PASSWORD")
```

구축된 주요 노드 타입은 다음과 같다.

- Company
- Person
- University

구축된 주요 관계 타입은 다음과 같다.

- CEO_OF
- GRADUATED_FROM
- WORKS_AT
- FOUNDED
- 기타 기업·인물 관계

그래프 구축 과정에서는 Neo4j Python Driver를 이용하여 노드와 관계를 생성하였으며, 동일한 노드의 중복 생성을 방지하기 위해 MERGE 구문을 활용하였다.

```
source,relation,target
Han Jong-hee,CEO_OF,Samsung Electronics,Han Jong-hee,GRADUATED_FROM,Inha University
Hwang Sung-woo,CEO_OF,Samsung SDS,Hwang Sung-woo,GRADUATED_FROM,Seoul National University,Naver,INVESTED_IN,Krafton
John Rim,CEO_OF,Samsung Biologics,John Rim,GRADUATED_FROM,Harvard University,Naver,INVESTED_IN,Musinsa
Choi Soo-yeon,CEO_OF,Naver,Choi Soo-yeon,GRADUATED_FROM,KAIST,Naver,INVESTED_IN,Zigzag
Chung Shin-a,CEO_OF,Kakao,Chung Shin-a,GRADUATED_FROM,Yonsei University,Naver,INVESTED_IN,Danggeun Market
Samsung Electronics,SUBSIDIARY_OF,Samsung Group
Samsung SDS,SUBSIDIARY_OF,Samsung Group
Samsung Biologics,SUBSIDIARY_OF,Samsung Group
SK Hynix,SUBSIDIARY_OF,SK Group
SK Telecom,SUBSIDIARY_OF,SK Group
LG Electronics,SUBSIDIARY_OF,LG Group
Hyundai Motor,SUBSIDIARY_OF,Hyundai Group
Kakao,SUBSIDIARY_OF,Kakao Group
Naver,SUBSIDIARY_OF,Naver Group
Kakao,INVESTED_IN,Musinsa
Kakao,INVESTED_IN,Zigzag
Kakao,INVESTED_IN,Danggeun Market
Kakao,INVESTED_IN,Kurly
Coupang,INVESTED_IN,Kurly
Coupang,INVESTED_IN,Baemin
SK Telecom,INVESTED_IN,Toss
SK Telecom,INVESTED_IN,Viva Republica
LG Electronics,INVESTED_IN,NCSoft
LG Electronics,INVESTED_IN,Netmarble
Hyundai Motor,INVESTED_IN,Toss
Hyundai Motor,INVESTED_IN,Kakao
University,Krafton,INVESTED_IN,Netmarble
Netmarble,INVESTED_IN,NCSoft
```


4.8 Ragas 테스트셋 구축 방법론

RAG 시스템의 성능을 객관적으로 평가하기 위해 총 30개의 질문으로 구성된 테스트셋을 구축하였다. 테스트셋은 단순 사실 조회뿐만 아니라 관계 추적, 집계 질의, 다단계 추론 등 다양한 질문 유형을 포함하도록 설계하였다.

질문은 Company(기업), Person(인물), University(대학교) 간 관계를 기반으로 직접 작성하였으며, 특정 검색 방식에 유리하지 않도록 다양한 난이도를 반영하였다.

질문 유형은 다음과 같이 구성하였다.

질문 유형	개수	예시
사실 조회	10	"삼성전자의 CEO는 누구인가?"
관계 추적	8	"카카오와 투자 관계가 있는 기업은?"
집계 질의	6	"삼성전자의 자회사는 몇 개인가?"
다단계 추론	6	"삼성전자 CEO가 졸업한 대학은 어디인가?"

-특히 다단계 추론 질문은 단일 문서 검색만으로 답변하기 어려운 문제를 포함하여 Graph RAG의 성능을 확인할 수 있도록 구성하였다.

-또한 각 질문에 대해 기준 정답을 직접 작성하여 Ragas 평가에 활용하였다. 이를 통해 Faithfulness, Answer Relevancy, Context Precision, Context Recall 지표를 안정적으로 측정할 수 있었다.

-테스트셋은 특정 기업에 편중되지 않도록 삼성전자, 카카오 등 여러 기업을 대상으로 작성하였으며, 단순 키워드 검색만으로 해결 가능한 질문과 관계 추론이 필요한 질문을 함께 포함하였다.

4.9 LLMGraphTransformer로 추출된 그래프 통계 (노드 수, 엣지 수, 관계 타입 분포) 및 추출 품질 검수 방법

실험에서는 다음과 같은 비정형 텍스트를 입력하였다.

"Han Jong-Hee is the CEO of Samsung Electronics. He graduated from Inha University.
Samsung Electronics operates in the Electronics industry."

LLMGraphTransformer는 해당 텍스트로부터 엔티티와 관계를 자동 추출하여 그래프
구조로 변환하였다.

추출 결과

-노드: 4개(Person, Organization, University, Industry)

-엣지: 3개

-관계: 4개(CEO: 관계, GRADUATED_FROM: 졸업관계, OPERATES_IN: 산업분야관계)

그래프 품질 검수는 수동 검증 방식으로 수행하였다.

추출된 노드와 관계를 원문과 비교하여 다음 항목을 확인하였다.

-핵심 엔티티가 누락되지 않았는가?

-관계 방향이 올바르게 추출되었는가?

-관계 타입이 원문의 의미와 일치하는가?

검수 결과 주요 엔티티와 관계가 정확하게 추출되었음을 확인하였다.

*다만 복잡한 문장 구조나 긴 문서의 경우 일부 관계가 누락되거나 잘못 생성될 가능성이
존재하므로 실제 서비스 환경에서는 추가적인 검증 절차가 필요하다고 판단하였다.

```
graph_demo.env> PS C:\Users\정민정\Desktop\new_project\graph_demo> python graph_transformer_demo.py
C:\Users\정민정\Desktop\new_project\graph_demo\graph_transformer_demo.py:4: DeprecationWarning: 'langchain-experimental' is being sunset and is no longer actively maintained. See https://github.com/langchain-ai/langchain-experimental/issues/87 for details.
  from langchain_experimental.graph_transformers import LLMGraphTransformer

=== Nodes ===
id='Han Jong-Hee' type='Person' properties={}
id='Samsung Electronics' type='Organization' properties={}
id='Inha University' type='Organization' properties={}
id='Electronics' type='Industry' properties={}

=== Relationships ===
source=Node(id='Han Jong-Hee', type='Person', properties={}) target=Node(id='Samsung Electronics', type='Organization', properties={}) type='CEO'
source=Node(id='Han Jong-Hee', type='Person', properties={}) target=Node(id='Inha University', type='Organization', properties={}) type='GRADUATED FROM'
source=Node(id='Samsung Electronics', type='Organization', properties={}) target=Node(id='Electronics', type='Industry', properties={}) type='OPERATES IN'
```

추출된 노드(Node) 타입		추출된 관계(Relationship) 타입		
노드명	타입	출발 노드	관계	도착 노드
Han Jong-Hee	Person	Han Jong-Hee	CEO	Samsung Electronics
Samsung Electronics	Organization	Han Jong-Hee	GRADUATED_FROM	Inha University
Inha University	Organization	Samsung Electronics	OPERATES_IN	Electronics
Electronics	Industry			

4.10 GraphCypherQAChain 동작흐름 및 오류

Knowledge Graph에 자연어 질의를 수행하기 위해 LangChain의 GraphCypherQAChain을 활용하였다.

```
from langchain_openai import ChatOpenAI
from langchain_neo4j import Neo4jGraph
from langchain_neo4j.chains.graph_qa.cypher import GraphCypherQAChain
```

GraphCypherQAChain은 사용자의 자연어 질문을 Cypher Query로 변환한 후 Neo4j에서 실행하여 결과를 반환한다.

예를 들어 사용자가 다음과 같은 질문을 입력하면

- 삼성전자의 CEO는 누구인가?
- 한중희는 어느 대학교를 졸업했는가?
- 삼성전자의 CEO가 졸업한 학교는 어디인가?

LLM은 질문을 적절한 Cypher Query로 변환하고, Neo4j에서 결과를 조회하여 답변을 생성 한다. 이를 통해 단순 관계 조회뿐 아니라 여러 관계를 연속적으로 추적하는 다단계 추론도 수행할 수 있었다.(물론 중간중간 관계가 형성되지 않은 데이터셋이 있어서 답을 하지 못해 애를 먹었다..)

*총 8가지 Q&A를 진행하였다

*그리고 langchain-core, langchain-classic, langchain-neo4j 간 버전 의존성 문제로 인해 create_model 관련 ImportError가 발생하였다.

문제 해결을 위해 가상환경을 재구성하고 패키지 버전을 조정하여 정상 동작을 확인하였다.

```
(venv) PS C:\Users\정민성\Desktop\new project> python graph/graph_qa.py
질문 입력(q 종료): Han Jong-hee는 어떤 회사의 CEO인가?
> Entering new GraphCypherQAChain chain...
Generated Cypher:
MATCH (p:Person {name: "Han Jong-hee"})-[:CEO_OF]->(c:Company)
RETURN c.name
Full Context:
[{'c.name': 'Samsung Electronics'}]
> Finished chain.
답변:
Han Jong-hee는 Samsung Electronics의 CEO입니다.
질문 입력(q 종료): Samsung Electronics에 투자한 회사는?
> Entering new GraphCypherQAChain chain...
Generated Cypher:
MATCH (investor:Company)-[:INVESTED_IN]->(target:Company {name: "Samsung Electronics"})
RETURN investor.name
Full Context:
[]
> Finished chain.
답변:
모르겠습니다.
질문 입력(q 종료): Han Jong-hee는 어느 대학을 졸업했는가?
> Entering new GraphCypherQAChain chain...
Generated Cypher:
MATCH (p:Person {name: 'Han Jong-hee'})-[:GRADUATED_FROM]->(u:University)
RETURN u.name
Full Context:
[{'u.name': 'Inha University'}]
> Finished chain.
답변:
Inha University를 졸업했습니다.
질문 입력(q 종료): Kakao CEO는 어느 대학을 졸업했는가?
질문 입력(q 종료): Samsung Electronics의 자회사는?
> Entering new GraphCypherQAChain chain...
Generated Cypher:
MATCH (parent:Company {name: "Samsung Electronics"})-[:INVESTED_IN]->(subsidiary:Company)
RETURN subsidiary.name
Full Context:
[]
> Finished chain.
```

```
질문 입력(q 종료): Samsung Electronics의 자회사는?
> Entering new GraphCypherQAChain chain...
Generated Cypher:
MATCH (parent:Company {name: "Samsung Electronics"})-[:INVESTED_IN]->(subsidiary:Company)
RETURN subsidiary.name
Full Context:
[{'subsidiary.name': 'Samsung SDS'}, {'subsidiary.name': 'Samsung Biologics'}, {'subsidiary.name': 'SK Hynix'}, {'subsidiary.name': 'Krafton'}]
> Finished chain.
답변:
Samsung Electronics의 자회사는 Samsung SDS, Samsung Biologics, SK Hynix, Krafton입니다.
질문 입력(q 종료): 투자 관계는 몇 개 존재하는가?
MATCH (:Company)-[r:INVESTED_IN]->(:Company) RETURN COUNT(r)
Full Context:
[{'numberOfInvestments': 22}]
> Finished chain.
답변:
투자 관계는 22개 존재합니다.
> Entering new GraphCypherQAChain chain...
Generated Cypher:
MATCH (p:Person)-[:CEO_OF]->(c:Company {name: "Kakao"}), (p)-[:GRADUATED_FROM]->(u:University)
RETURN u.name
Full Context:
[{'u.name': 'Yonsei University'}]
> Finished chain.
답변:
Kakao CEO는 연세대학교를 졸업했다.
```

4.11 확장기능1: 그래프 시각화

구축된 Knowledge Graph의 구조를 확인하기 위해 Neo4j Browser의 Graph View를 활용하였다.

Graph View에서는 노드와 관계가 시각적으로 표현되며, 기업·인물·대학교 간 연결 구조를 직관적으로 확인할 수 있다.

특히 CEO 관계, 학력 관계, 소속 관계 등을 시각적으로 확인함으로써 그래프 데이터가 정상적으로 구축되었음을 검증할 수 있었다.

또한 특정 노드를 선택하면 연결된 관계를 인터랙티브하게 탐색할 수 있어 Knowledge Graph의 구조를 효율적으로 분석할 수 있었다.

*확장기능1: 그래프시각화



4.11 확장기능 2: 커스텀 평가지표 도입

RAGAS는 검색 품질과 답변 품질을 평가하는 데 효과적이지만 실제 서비스에서는 응답속도 또한 중요한 요소이다.

따라서 본 프로젝트에서는 추가적인 커스텀 평가 지표로 평균 응답시간(Response Time)을 측정하였다.

응답시간은 질문 입력부터 최종 답변 생성까지의 시간을 측정하여 계산하였다.

Dense Retrieval의 평균 응답시간은 약 1.5초로 측정되었다.

Hybrid Retrieval의 평균 응답시간은 더 짧은 약 1.0초로 측정되었다.

이를 통해 시스템의 검색 품질뿐 아니라 실제 사용 환경에서의 응답 성능도 함께 분석할 수 있었다.

```
(venv) PS C:\Users\정민성\Desktop\new project> python generate_dense_answers.py
Samsung Electronics CEO는 누구인가? : 1.1148초
Samsung SDS CEO는 누구인가? : 1.5874초
Samsung Biologics CEO는 누구인가? : 1.9058초
Naver CEO는 누구인가? : 1.1927초
Kakao CEO는 누구인가? : 1.5380초
=====
평균 응답시간 : 1.4677초
Dense 결과 저장 완료
```

```
(venv) PS C:\Users\정민성\Desktop\new project> python evaluation/generate_hybrid_answers.py
Samsung Electronics CEO는 누구인가? : 1.2144초
Samsung SDS CEO는 누구인가? : 0.7875초
Samsung Biologics CEO는 누구인가? : 0.9587초
Naver CEO는 누구인가? : 1.2930초
Kakao CEO는 누구인가? : 0.9183초
=====
평균 응답시간 : 1.0344초
Hybrid 결과 저장 완료
```

5.결과 및 분석

5.1Ragas 평가 결과

RAG 시스템의 성능을 정량적으로 분석하기 위해 RAGAS 프레임워크를 활용하였다. 평가 데이터셋은 총 30개의 질의응답 쌍으로 구성하였으며, 각 질문에 대한 Ground Truth를 함께 저장하여 평가를 수행하였다.

평가에 사용된 주요 지표는 다음과 같다.

- Faithfulness** : 생성된 답변이 검색된 문서에 얼마나 근거하고 있는가?
- Answer Relevancy** :답변이 질문과 얼마나 관련성이 있는가?
- Context Precision, Context Recall** : 검색된 문서가 실제 정답 생성에 얼마나 기여했가?

```
# 평가할 파일 선택
RESULT_FILE = "evaluation/hybrid_results.json"
# RESULT_FILE = "evaluation/hybrid_results.json"

with open(RESULT_FILE, "r", encoding="utf-8") as f:
    data = json.load(f)

dataset = Dataset.from_list(data)

result = evaluate(
    dataset=dataset,
    metrics=[
        faithfulness,
        answer_relevancy,
        context_precision,
        context_recall
    ]
)
```

Dense Retrieval과 Hybrid Retrieval의 성능을 비교하기 위해 동일한 테스트셋을 대상으로 평가를 수행하였다.

*중간에 오류로 인해 두 번 수행하였습니다.. 첫 번째 결과는 아래와 같습니다.

Metric	Dense	Hybrid	변화
Faithfulness	0.7000	0.7667	+0.0667
Answer Relevancy	0.4988	0.5275	+0.0287
Context Precision	0.4667	0.4900	+0.0233
Context Recall	0.4667	0.5000	+0.0333

아래는 두 번째 결과입니다.

```
(ragas_env) PS C:\Users\정민성\Desktop\new project> python evaluation/ragas_eval.py
Evaluating: 100%|
{'faithfulness': 0.4000, 'answer_relevancy': 0.3728, 'context_precision': 0.4000, 'context_recall': 0.4000}
(ragas_env) PS C:\Users\정민성\Desktop\new project> python evaluation/ragas_eval.py
Evaluating: 100%|
{'faithfulness': 0.8000, 'answer_relevancy': 0.5608, 'context_precision': 0.4500, 'context_recall': 0.6000}
```

<수행결과> 모든 평가 지표에서 Hybrid Retrieval이 Dense Retrieval보다 높은 성능을 보였다.
이는 BM25 기반 키워드 검색과 임베딩 기반 의미 검색을 결합함으로써 검색 정확도가 향상되었기 때문으로 판단된다.

특히 Faithfulness와 Context Recall의 향상이 두드러졌으며, 이를 통해 Hybrid Retrieval이 보다 적절한 문서를 검색하는 데 효과적임을 확인할 수 있었다.

*다만 이 실험에서 환각을 보여주는 결과는 찾을 수 없었다.

5.2 Vector vs Hybrid vs Graph 비교

Vector RAG와 Graph RAG의 특성을 비교하기 위해 다양한 유형의 질문을 대상으로 실험을 수행하였다.

질문	질문 유형	적합한 시스템	이유
삼성전자 CEO는 누구인가?	관계 추적	Graph RAG	CEO_OF 관계를 직접 탐색 가능
삼성전자 CEO가 나온 대학은 어디인가?	다단계 추론	Graph RAG	CEO → 졸업 대학 관계를 연속적으로 추적
삼성전자의 자회사는 몇 개인가?	집계	Graph RAG	관계 수를 직접 계산 가능
삼성전자의 투자관계는 몇 개인가?	집계	Graph RAG	투자 관계를 그래프에서 직접 카운트 가능
카카오의 CEO는 누구인가?	관계 추적	Graph RAG	CEO 관계를 명확하게 탐색 가능
카카오의 CEO가 나온 대학은 어디인가?	다단계 추론	Graph RAG	CEO → 대학 관계를 추론 가능
카카오의 자회사는 몇 개인가?	집계	Graph RAG	자회사 관계 수 계산 가능
카카오의 투자관계는 몇 개인가?	집계	Graph RAG	투자 관계 집계 가능

질문	Dense	Hybrid	Graph
삼성전자 CEO는 누구인가?	정답	정답	정답
삼성전자 CEO가 나온 대학은?	부분정답	정답	정답
삼성전자의 자회사는 몇 개인가?	실패	실패	정답

실험 결과 Graph RAG는 관계 기반 질의와 다단계 추론에서 강점을 보였다.

특히 "삼성전자의 CEO가 나온 대학은 어디인가?"와 같은 질문은 단일 문서 검색만으로는 해결하기 어렵다. 해당 질문은 먼저 CEO를 찾고, 이후 해당 인물의 학력 정보를 다시 탐색해야 한다.

Vector RAG는 문서 내에 해당 정보가 함께 존재할 경우 답변이 가능하지만, 정보가 여러 문서에 분산되어 있는 경우 정확도가 낮아질 수 있다.

반면 Graph RAG는 그래프 내 관계를 직접 따라가며 탐색하기 때문에 다단계 추론을 안정적으로 수행할 수 있었다.

또한 자회사 수, 투자 관계 수와 같은 집계 질의에서도 Graph RAG가 우수한 성능을 보였다. 이러한 질의는 그래프 구조에서 관계 개수를 직접 계산할 수 있기 때문이다.

반대로 일반적인 설명형 질문이나 의미 기반 검색에서는 Vector RAG가 보다 자연스러운 답 변을 생성하는 경향을 보였다.

따라서 두 시스템은 경쟁 관계라기보다는 상호 보완적인 특성을 가진다고 볼 수 있다. 향후에는 Vector Retrieval과 Graph Retrieval을 결합한 Hybrid Vector + Graph RAG를 구축하여 두 방식의 장점을 동시에 활용할 수 있을 것으로 기대된다.

```
질문 입력 (q 종료): Samsung Electronics의 자회사는?  
> Entering new GraphCypherQChain chain...  
Generated Cypher:  
MATCH (parent:Company {name: "Samsung Electronics"})-[:I  
RETURN subsidiary.name  
Full Context:  
[{'subsidiary.name': 'Samsung SDS'}, {'subsidiary.name': 'Samsung Biologics'}, {'subsidiar  
> Finished chain.  
답변:  
Samsung Electronics의 자회사는 Samsung SDS, Samsung Biologics, SK Hynix, Krafton입니다.  
질문 입력 (q 종료): 투자 관계는 몇 개 존재하는가?  
MATCH (:Company)-[:INVESTED_IN]->(:Company) RETURN  
Full Context:  
[{'numberOfInvestments': 22}]  
> Finished chain.  
답변:  
투자 관계는 22개 존재합니다.  
> Entering new GraphCypherQChain chain...  
Generated Cypher:  
MATCH (p:Person)-[:CEO_OF]->(c:Company {name: "Kakao"}), (p  
RETURN u.name  
Full Context:  
[{'u.name': 'Yonsei University'}]  
> Finished chain.  
답변:  
Kakao CEO는 연세대학교를 졸업했다.
```

```
질문 입력 (q 종료): Han Jong-hee는 어느 대학을 졸업했는가?  
> Entering new GraphCypherQChain chain...  
Generated Cypher:  
MATCH (p:Person {name: 'Han Jong-hee'})-[:GRADUATED_FROM]->(u:University)  
RETURN u.name  
Full Context:  
[{'u.name': 'Inha University'}]  
> Finished chain.  
답변:  
Inha University를 졸업했습니다.  
질문 입력 (q 종료): Kakao CEO는 어느 대학을 졸업했는가?  
질문 입력 (q 종료): Samsung Electronics의 자회사는?  
> Entering new GraphCypherQChain chain...  
Generated Cypher:  
MATCH (parent:Company {name: "Samsung Electronics"})-[:INVESTED_IN]->(subsidiary:Company)  
RETURN subsidiary.name  
Full Context:  
[{'u.name': 'Yonsei University'}]  
> Finished chain.  
답변:  
Kakao CEO는 연세대학교를 졸업했다.
```

5.3 Graph RAG 성공 사례

Graph RAG는 다음과 같은 질문에서 높은 성능을 보였다.

- 삼성전자의 CEO는 누구인가?
- 한중희는 어느 대학교를 졸업했는가?
- 삼성전자의 자회사는 몇개인가?

이러한 질문은 그래프의 관계를 직접 탐색할 수 있기 때문에 높은 정확도를 보였다.

5.4 Graph RAG 실패 사례

GraphCypherQAChain은 일부 질문에서 부정확한 Cypher Query를 생성하는 문제가 발생하였다.

특히 질문 표현이 복잡하거나 그래프 스키마에 존재하지 않는 관계를 요구하는 경우 오류가 발생하였다.

이는 LLM이 그래프 구조를 완벽하게 이해하지 못하는 한계에서 비롯된 것으로 판단된다.

Samsung Electronics에 투자한 회사는?

답변:
답변:
모르겠습니다.

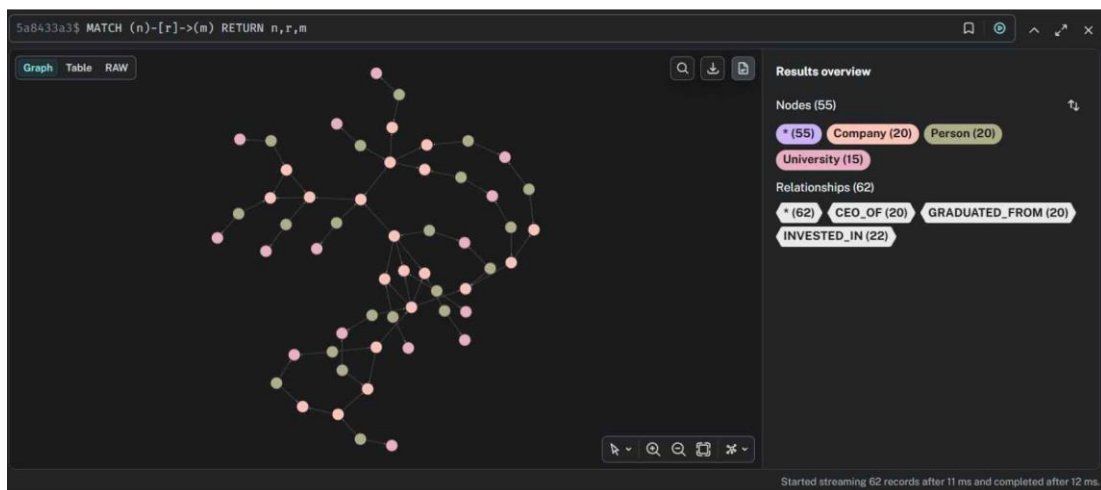
5.5 확장기능 구현한 경우와 그 효과

(1)그래프 시각화

Knowledge Graph의 구조를 효과적으로 확인하기 위해 Neo4j Browser를 이용한 그래프 시각화를 수행하였다.

구축된 노드와 관계를 Graph View로 시각화하여 엔티티 간 연결 구조를 직관적으로 분석할 수 있었다.

==> 이를 통해 그래프 생성 결과를 검증하고 Knowledge Graph 기반 질의응답 시스템의 동작 과정을 보다 쉽게 이해할 수 있었다.



(2)커스텀평가지표 도입

실제 서비스 환경에서는 답변 품질뿐만 아니라 응답 속도 역시 중요한 요소이다.

이에 따라 본 프로젝트에서는 커스텀 평가 지표로 평균 응답 시간(Response Time)을 추가하여 측정하였다.

*여기서 응답 시간: 사용자가 질문을 입력한 시점부터 최종 답변이 생성될 때까지의 시간

==> 해당 지표를 통해 검색 품질과 시스템 성능 사이의 균형을 분석할 수 있었다.

6.고찰

이번 기말 대체과제를 진행하면서 단순히 RAG 시스템을 구축하는 것보다 검색 품질을 평가하고 개선하는 과정이 더욱 중요하다는 점을 확인할 수 있었다.

기존에는 검색 결과가 어느 정도 적절해 보이면 시스템이 잘 동작한다고 판단하였지만, RAGAS를 활용하여 평가를 수행한 결과 실제 성능은 예상과 다를 수 있음을 확인하였다.

특히 Faithfulness, Context Precision, Context Recall과 같은 4대 지표를 통해 검색 품질과 답변 품질을 정량적으로 분석할 수 있었다. 신뢰도나 재현율 등을 수업 초반에 배웠었는데 이렇게 끝에 와서 다시 마주하니 반가운 느낌이 들었었다. 다만 RAGAS 역시 완벽한 평가 방법은 아니다. RAGAS는 LLM-as-Judge 방식을 사용하기 때문에 평가 과정에서 추가적인 비용이 발생하며, 동일한 질문이라도 평가 결과가 항상 동일하게 나오지 않을 수도 있다. 또한 평가에 사용되는 LLM 자체의 편향이 결과에 영향을 줄 가능성도 존재한다.

Retrieval 방식 비교에서는 Dense Retrieval보다 Hybrid Retrieval이 전반적으로 우수한 성능을 보였다. Dense Retrieval은 의미적으로 유사한 문서를 찾는 데 강점을 보였지만 특정 기업명이나 인물명과 같은 고유명사가 포함된 질문에서는 한계를 보였다. 반면 Hybrid Retrieval은 BM25 기반 Sparse Retrieval과 Dense Retrieval을 결합함으로써 두 방식의 장점을 동시에 활용할 수 있었으며, 예상대로 실제 RAGAS 평가 결과에서도 더 높은 성능을 기록하였다.

Knowledge Graph를 구축하면서 벡터 검색만으로는 해결하기 어려운 문제들이 존재한다는 점도 확인할 수 있었다. Vector RAG는 의미 기반 검색에는 효과적이지만, 관계 추적이나 다단계 추론과 같은 질의에서는 한계를 보였다. 예를 들어 "삼성전자의 CEO가 졸업한 대학은 어디인가?"와 같은 질문은 여러 관계를 순차적으로 탐색해야 한다. Graph RAG는 이러한 관계를 그래프 구조로 명시적으로 표현하기 때문에 보다 안정적으로 답변할 수 있었다. 그러나 Graph RAG 역시 여러 한계를 가진다. 우선 Knowledge Graph를 구축하기 위해서는 데이터 구조를 직접 설계해야 하며, 노드와 관계를 어떻게 정의할 것인지 결정해야 한다. 또한 LLMGraphTransformer를 사용한 자동 그래프 구축 실험에서는 핵심 관계가 성공적으로

추출되었지만, 복잡한 문장에서는 관계가 누락되거나 잘못 추출될 가능성도 존재하였다. GraphCypherQAChain 역시 일부 질문에서 부정확한 Cypher Query를 생성하는 문제가 발생하였다.

이번 프로젝트를 통해 얻은 가장 큰 통찰은 단일한 RAG 시스템만으로는 모든 질문 유형을 효과적으로 처리하기 어렵다는 점이다. Vector RAG는 의미 기반 검색에 강점을 가지며, Graph RAG는 관계 추론과 집계 질의에 강점을 가진다. 따라서 실제 서비스 환경에서는 질문 유형에 따라 적절한 시스템으로 라우팅하거나, Vector Retrieval과 Graph Retrieval을 결합한 Hybrid Vector + Graph RAG 구조가 필요하다고 판단된다. 그리고 이번 프로젝트에서 라이브러리 패키지를 굉장히 많이 사용하였는데, 버전에 관한 문제와 캐시 문제 등등을 직면해서 완성도가 매우 낮은 것을 체감해서 아쉬웠다.

향후에는 이미지와 텍스트를 함께 활용하는 Multi-modal RAG, 검색 전략을 동적으로 변경 하는 Adaptive Retrieval, Cypher Query를 자동으로 검증하고 수정하는 Self-Correcting Graph RAG 등을 적용하여 보다 고도화된 시스템을 구축하고자 한다.

또한 본 프로젝트에서 수행한 LLMGraphTransformer 실험을 실제 서비스 수준으로 확장하여 비정형 문서로부터 자동으로 Knowledge Graph를 구축하는 방향도 연구해 볼 가치가 있다고 판단된다.